Skill-20

Name: k.vaishali

Rollno: 2520030117

Section: 8

Aim:

To capture SIGTSTP, suspend processes, update the job table, preserve process state, resume jobs later, and test the suspension workflow. Also, to create process groups, assign group IDs, transfer terminal control, restore terminal ownership, coordinate signals, and validate the behavior.

Objective:

- To capture the SIGTSTP signal generated by Ctrl+Z.
- To suspend a running foreground process.
- To preserve the state of the suspended process.
- To maintain and update a job table.
- To resume a suspended process later.
- To create and manage process groups.
- To assign process group IDs using setpgid().
- To transfer terminal control using tcsetpgrp().
- To restore terminal ownership to the shell.
- To coordinate signals between the shell and process

groups. • To test and validate process suspension and

resumption. **Problem Statement:**

Design and implement a shell-style job-control mechanism in C that handles SIGTSTP generated by Ctrl+Z. The implementation

should create a child process group, suspend the foreground process,
preserve its state, maintain the job state, transfer terminal control when required, restore terminal ownership to the shell, and resume the suspended process later.

Theory:

1. SIGTSTP

SIGTSTP is a signal normally generated when the user presses Ctrl+Z in a terminal. It requests that the running foreground process be suspended.

2. Process Suspension

When a process receives SIGTSTP, it can be stopped temporarily. Its current execution state is preserved so that it can continue later.

3. Process Groups

A process group is a collection of related processes. Process groups are useful in shell job control because signals can be sent to all processes belonging to a particular job.

4. Process Group ID

Each process group has a process group ID. The setpgid() system call can be used to place a process into a process group.

5. Terminal Control

A terminal has a foreground process group. The tcsetpgrp() system call can transfer terminal control to a particular process group.

6. Restoring Terminal Ownership

After a foreground job is suspended or completed, the shell can regain terminal control by using tcsetpgrp() with the shell's process group ID.

7. Job Table

A shell can maintain a job table containing information such as process ID, process group ID, and current job state.

8. Job States

A job may be in states such as **Running**, **Stopped**, or **Completed**. When SIGTSTP stops a process, its job state can be updated to Stopped.

9. Resuming a Job

A stopped process can later be continued by sending SIGCONT. The shell can use kill() to send this signal to the process group.

10. Signal Coordination

Signals such as SIGTSTP and SIGCONT can be coordinated between the shell and foreground process group to provide job-control behavior.

Algorithm:

1. Start the program.
2. Create a child process using fork().
3. Create a separate process group for the child.
4. Assign the child process group ID using setpgid().
5. Transfer terminal control to the child process group. 6. Execute the foreground command.
7. Wait for the child process using waitpid().
8. Detect whether the child was stopped by SIGTSTP. 9. Update the job table with the Stopped state.
10. Restore terminal control to the shell.
11. Display the suspended job information.

12. Send SIGCONT to the stopped process group.

13. Transfer terminal control back to the resumed process group. 14. Wait for the process to finish.

15. Restore terminal control to the shell.

16. Update the job state to Completed.

17. Stop the program.

Data Used

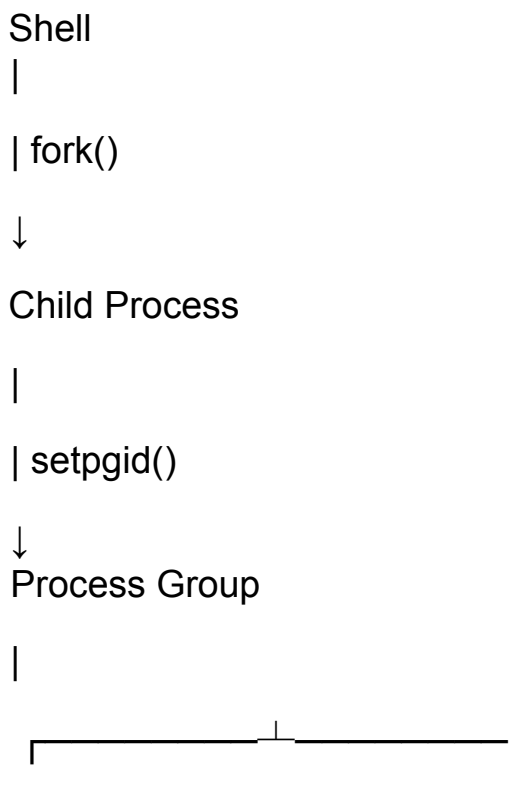
- Process ID (pid_t)
- Process Group ID (pid_t)
- Job state
- Process status
- Terminal file descriptor
- struct sigaction

Operations Used

- fork() for creating a child process.
- setpgid() for creating and assigning process groups. • getpgrp() for obtaining the shell process group.
- tcsetpgrp() for transferring terminal control.
- waitpid() for monitoring the child process.
- kill() for sending SIGCONT.
- SIGTSTP for suspending a process.
- SIGCONT for resuming a suspended process. •

WIFSTOPPED() for checking whether a process was stopped. • WSTOPSIG() for identifying the stopping signal. • WIFEXITED() for checking normal completion. **Pipeline /**

Process Group Structure:



Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
```

```
#include <sys/wait.h>


int main() {

 pid_t pid, shell_pgid;

 int status;
 printf("Skill - 20: SIGTSTP and Job

Control\n");

printf("---------------------------------\n");


 shell_pgid = getpgrp();


 pid = fork();
 if (pid < 0) {

 perror("fork");

 return 1;

 }


 if (pid == 0) {

 setpgid(0, 0);


 signal(SIGTSTP, SIG_DFL);

 signal(SIGINT, SIG_DFL);
```

```
 printf("Foreground job started. Press Ctrl+Z to suspend.\n");


execlp("sleep", "sleep", "20", NULL);



 perror("sleep");
 exit(1);

 }


 setpgid(pid, pid);


 tcsetpgrp(STDIN_FILENO, pid);


 waitpid(pid, &status, WUNTRACED);
 tcsetpgrp(STDIN_FILENO, shell_pgid);


 if (WIFSTOPPED(status)) {

 printf("Job state: Stopped by signal %d\n",

WSTOPSIG(status));


 printf("Job suspended. Resuming


job...\n"); kill(-pid, SIGCONT);
```

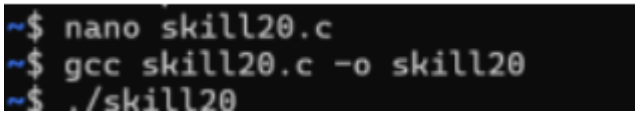
```
tcsetpgrp(STDIN_FILENO, pid);

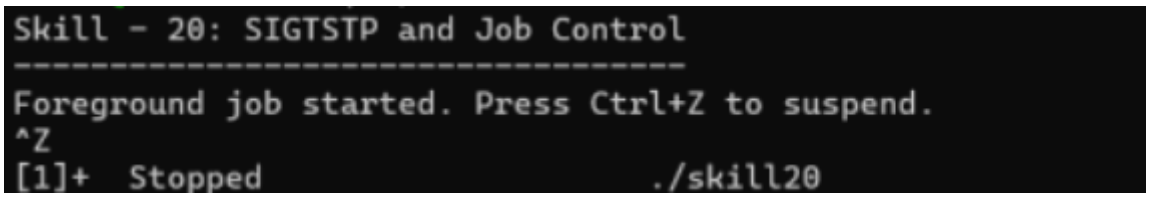

waitpid(pid, &status, 0);

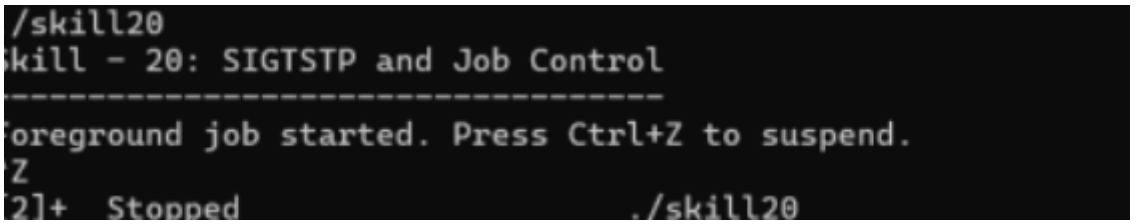

tcsetpgrp(STDIN_FILENO, shell_pgid);
if (WIFEXITED(status)) {
printf("Job state: Completed normally\n"); }
} else if (WIFEXITED(status)) {
printf("Job state: Completed normally\n"); }


printf("Job control test completed.\n");
return 0;
}
```

Output:







Program Explanation:

16.1 Process Creation

The fork() system call creates a child process that acts as the foreground job.

16.2 Process Group Creation

The child uses:

setpgid(0, 0);
This creates a separate process group for the child.

16.3 Terminal Control

The parent uses:

tcsetpgrp(STDIN_FILENO, pid);

to give terminal control to the foreground process

group. **16.4 SIGTSTP**

When the user presses Ctrl+Z, the terminal sends SIGTSTP to the foreground process group. The process becomes stopped.

16.5 Detecting Suspension

The parent uses:

waitpid(pid, &status, WUNTRACED);

WUNTRACED allows the parent to receive information when the child is stopped.

16.6 Updating Job State

The following condition checks whether the process was

stopped: WIFSTOPPED(status)

If true, the job state is displayed as Stopped.

16.7 Restoring Terminal Control

After the job is stopped, terminal control is returned to the

shell: tcsetpgrp(STDIN_FILENO, shell_pgid);

16.8 Resuming the Job

The stopped process group is resumed using:

kill(-pid, SIGCONT);

The negative PID causes the signal to be sent to the process group. **16.9 Final Job State**

After resuming, waitpid() waits for the process to complete. The program then displays the final job state.

Testing:

- Tested creation of a child process.
- Tested creation of a separate process group.
- Tested terminal control transfer.
- Tested SIGTSTP using Ctrl+Z.
- Verified that the foreground process was suspended.
- Verified that the job state changed to Stopped.
- Tested restoration of terminal control to the shell.
- Tested resumption using SIGCONT.
- Verified that the resumed process completed

successfully. • Tested final job-state reporting.

Test Cases:

Test Case	Input / Action	Expected Result
1	Run the program	Foreground job starts.

2	Press Ctrl+Z	Foreground job is suspended.
3	Check job status	Job state is displayed as Stopped.
4	Restore terminal control	Shell receives terminal control.

5 Send SIGCONT Suspended job resumes.

6	Wait for completion	Job completes normally.
7	Check process group	Child belongs to its own process group.

Observation:

- The child process was created successfully.
- A separate process group was created for the foreground job.
- Terminal control was transferred to the foreground process group.
- Ctrl+Z generated SIGTSTP and suspended the foreground process.
- The stopped process retained its execution state.
- The job state was updated to Stopped.
- Terminal control was restored to the shell.
- SIGCONT successfully resumed the suspended process.
- The resumed process completed successfully.
- Process groups and signal handling worked as expected.

Result:

Process suspension and job-control mechanisms were successfully implemented. The program created a process group, transferred terminal control, captured the SIGTSTP suspension event, updated

the job state, restored terminal ownership, resumed the stopped process using SIGCONT, and verified its completion.

Conclusion:

The practical demonstrated process suspension and shell-style job control using SIGTSTP, SIGCONT, process groups, terminal control, and process status functions. It showed how a shell can suspend a foreground job, preserve its state, update its job table, restore terminal ownership, and resume the job later.